

Databricks Platform Security Docs

Question: How does Databricks secure Genie?



Table of Contents

Overview	3
How Genie works	3
How generation and execution are separated	3
The agentic loop	3
What data is used	4
Which models	4
Where processing happens and data residency	5
What is logged and retained	5
Shared design principles	5
Threat model	6
Unauthorized access by compromise of over-permissioned users	7
Sensitive data disclosure via Model Providers	8
Prompt injection, jailbreaks, or malicious retrieved content	9
External connector and shadow-AI risk	10
Static sharing and downstream disclosure of results	11
AI supply chain, model routing, and content integrity	12
Answer accuracy, hallucination, and overreliance	13
Control catalog	15
Genie accuracy and trust	15
AI-specific commitments	16
More Information	17
Revision history	17

Overview

Genie gives everyone in your organization a single place to ask questions of your data in natural language. A user asks a question in natural language, Genie generates SQL, runs it against your governed tables, and returns the result. Every answer is computed from your own data and governed by Unity Catalog.

This paper is written for security teams evaluating Genie for deployment. It explains how Genie works, what data is involved, and how the risks customers most often raise are bounded, threat by threat. Genie runs inside the Databricks architecture you have already secured. It introduces no new data plane, no separate permission model, and no path around Unity Catalog. If you have implemented the controls in [Security Best Practices](#) for Databricks, your Genie deployment already inherits that security posture. You do not need to run a separate threat-modeling exercise for Genie. We have done it for you, and this paper is the result.

There is one Genie-specific thing we ask you to do. Ground your Genie Agents well by curating the tables, metadata, instructions, and example queries in each Agent per our documentation. This is the single highest-leverage action for answer quality and trust, and it is work your data owners do, not your security team. If that is all you need, you can stop here. The rest of this paper is for teams who want the threat-by-threat detail — including “Answer accuracy, hallucination, and overreliance,” which answers the question security and data teams ask most: can I trust the answer?

This paper covers Genie Agents, the managed natural-language experience built into the Databricks platform. It does not cover Genie Code (formerly the Databricks Assistant); the controls here apply to it, but it carries additional considerations covered separately.

How Genie works

Understanding two things resolves most security questions about Genie. The first is how the loop operates. The second is what data leaves your governance boundary.

How generation and execution are separated

Genie separates generating a query from running it, and this separation is the foundation of its security model.

SQL generation is done by a large language model. We send the model your question, relevant table metadata, sampled values, prior results, and any errors. The model returns a SQL statement and explanatory text.

SQL execution happens on a Databricks SQL warehouse, exactly as it would if a user typed the query into the SQL editor. The query runs under the calling user's own identity, and Unity Catalog applies that user's permissions, row filters, and column masks to the result.

The consequence is that the model proposes, but Unity Catalog decides. A model cannot return a row the user was not already entitled to query, because authorization is enforced at the query engine, outside the model. This holds regardless of what the model was told or asked to do.

The agentic loop

Genie is iterative, not a single request and response. A typical exchange runs like this:

- The user asks a question in natural language.
- Genie assembles context, the question, table metadata, relevant values, and any Agent instructions, and sends it to the model.
- The model returns SQL.
- The SQL runs on the SQL warehouse under the user's identity, and Unity Catalog enforces their permissions.
- Results return to Genie, which may send them back to the model to summarize the answer or inform a follow-up.
- The user asks a follow-up, and the loop repeats, now with prior results as part of the context.

The fifth step is worth being explicit about. Query results can be sent back to the model to summarize an answer or to answer a follow-up such as "has that top seller always discounted more?" Those results are, by construction, data the user was already authorized to see.

What data is used

Data from your workspace is used in three categories. We apply the same protections to all three and do not vary controls by category or by source.

- **User inputs.** Free text the user types, plus the configuration the Agent author supplied, such as instructions, guidance, and example queries.
- **Metadata.** Pulled automatically when tables are added to an Agent, including column names, descriptions, and key relationships. Curated example queries may also be included.
- **Relevant values.** Actual values from your data, included so the Agent can build accurate queries. Prompt matching, formerly value sampling, lets the model reconcile a user asking for "Ford F150" with a stored value of "F-150". These values are held in a control plane database or workspace system data, and are covered by customer-managed keys where enabled. Prior results let follow-up questions and refinements reference the previous answer.

To make this concrete, consider an Agent over a car-sales table asked "Which dealerships are selling the most Ford F150s this quarter, and at what discount rate?" The user input is the question plus the author's guidance, such as "Q1 starts Feb 01". The metadata is the sales table's columns and relationships. The relevant values are sampled make and model values that let the model match "F150" to "F-150", and on the follow-up "has that #1 seller always discounted more?" the prior result set identifying that dealer.

Which models

The authoritative mapping of feature to model is the Databricks [AI Feature Trust and Safety page](#). As of June 2026, Genie Agents use the models shown below.

Feature	Model
Genie Code	Azure OpenAI service or OpenAI on Databricks
Genie Code Agent Mode	Azure OpenAI service, OpenAI on Databricks, or Anthropic on Databricks
Genie Code Autocomplete	Databricks-hosted model
AI-generated comments	Azure OpenAI service or Databricks-Hosted

AI/BI dashboard AI-assisted visualizations	Azure OpenAI service
Genie Agents	Azure OpenAI service
Genie Agents Agent Mode	Azure OpenAI service or Anthropic on Databricks
LLMs as judges*	Azure OpenAI service (or user-specified)
Synthetic data generation*	Azure OpenAI service

* These features are not AI assistive features, but require the partner-powered AI features setting.

From Databricks AI Feature Trust & Safety, Last updated July 2026

Customers do not select the model. We manage it. We continuously evaluate alternatives, including other frontier providers, and may move a feature to a better-performing model. Any model we use adheres to the same security model, with zero data retention and no training on your data.

Where processing happens and data residency

Genie orchestration runs in the Databricks control plane. Query execution runs on your SQL warehouse. Model inference runs on the Databricks-selected provider endpoint.

AI features are Designated Services and comply with [Databricks Geos data-residency boundaries](#). In the cases documented for each cloud, an AI feature may call beyond the local cloud or geo if cross-geo processing is enabled. Azure OpenAI, for example, is available in Azure regions but not in other clouds. When that happens, the request goes to a Databricks-owned LLM proxy in the source control plane, which sends it directly to the model in the destination region. The destination region performs no logging, maintains no state, and creates no record of the request.

What is logged and retained

Databricks understands the importance of your data and the trust you place in us when you use our platform and Databricks AI assistive features.

- **Model providers retain nothing.** Our partner-powered AI features use zero-data-retention endpoints. Providers do not retain your data, even for abuse monitoring, and neither we nor our partners train generative models on it.
- **Databricks retains operational logs for 30 days.** Inputs, meaning metadata and row-level data relevant to the question, and outputs, meaning generated prompt text, are logged and retained for 30 days for debugging and support.
- **Access to those logs is controlled.** Only Databricks staff whose job requires it can reach them, such as support, engineering, and security. Access is through an internal workspace requiring SSO, MFA, and VPN, requires internal approval, and is itself audit-logged into the Databricks SIEM.
- **Long-term application data is yours and encrypted.** The Genie Agents database, dashboards, and history are AES-256 bit encrypted at rest and are covered by customer-managed keys for managed services where you enable them.

Shared design principles

Genie reuses existing platform infrastructure rather than introducing parallel systems, so a single uniform security model applies. These guarantees hold for every Genie interaction.

- **On-behalf-of execution.** Every query runs on a SQL warehouse under the calling user's identity. Genie has no elevated identity and no standing credential of its own.
- **Governed by Unity Catalog.** One permission model, enforced end to end at the query engine, outside the model. The same privileges, row filters, and column masks apply as to a hand-written query. Data a user cannot access is never sent to a model.
- **Speed, not scope.** Genie changes how quickly a user can find and act on data they can already reach. It does not expand what anyone can reach.
- **Stateless models, no training.** The model stores nothing between requests, and each request carries only the conversation so far. Partner-powered features use zero-data-retention endpoints, so partners do not retain your data and we do not train generative foundation models on it.
- **Encrypted everywhere.** AES-256 or greater at rest and TLS 1.2 or greater in transit, with customer-managed keys available for long-term storage.
- **Usage data is protected and time-bounded.** Operational logs are retained 30 days, reachable only by staff whose role requires it, under just-in-time authorization and centralized audit.
- **Data residency is respected.** AI features are Designated Services. Cross-geo processing occurs only where enabled and documented, through a Databricks-owned proxy, with no logging or state in the destination region.
- **Controls are deterministic.** The security boundary does not depend on the model behaving well. Enforcement is applied by the platform, not chosen by the model.
- **Shared responsibility.** We enforce the platform boundary. You own identity, least privilege, network posture, and how results are handled downstream.

Threat model

These are the six security risks customers raise about Genie. Each is stated as the customer asks it, assessed directly, rated, and mapped to the specific controls that bound it.

Two layers of control appear throughout. Platform-inherited controls, numbered 1.x through 8.x, are capabilities you configure. Categories 1 through 7 are the same catalog as Security Best Practices for Databricks, and category 8 is Genie-specific and covered in the Control catalog. AI-specific commitments, numbered AI 1.0 through AI 9.0, are guarantees we enforce on every Genie interaction, independent of configuration.

#	Threat	Residual risk (properly configured)
1	Unauthorized access via a compromised or over-permissioned user	Low
2	Sensitive data disclosure to model providers	Low
3	Prompt injection, jailbreaks, or malicious retrieved content	Low
4	External connector and shadow-AI risk	Low

5	Static sharing and downstream disclosure of results	Low in-platform / Low to Moderate static
6	AI supply chain, model routing, and content integrity	Low

A seventh question, "can I trust that the answer is accurate?", comes up just as often. It is a quality question rather than a security one, and it is assessed in full in the section on answer accuracy below.

Unauthorized access by compromise of over-permissioned users

Concern	"does Genie give an attacker a faster way to find data if an account is compromised?"
Likelihood	Moderate
Impact	Moderate
Residual risk	Low
Bottom line	The fix is right-sizing entitlements and perimeter security, not withholding Genie. The exposure exists with or without Genie, and Genie only makes it faster to realize.

Risk description

AI can make it easier to find, summarize, and act on information a user can already reach. If an account is compromised or over-permissioned, that discovery gets faster.

Genie changes the speed of data discovery, not the scope of data access. Every query Genie generates runs on a SQL warehouse under the calling user's identity, and Unity Catalog applies that user's own permissions, row filters, and column masks to the result. Genie has no elevated identity and no path around Unity Catalog, so it cannot return a row the user could not already query. Exposure is bounded by what that identity was entitled to in the first place, with no privilege escalation and no cross-tenant reach.

Control Category	Control
Identity and access	1.1 and 1.2 SSO with MFA 1.3 AIM and SCIM sync 1.7 least privilege via groups 1.9 token management 1.14 the Genie interface as a least-privilege control.
Unity Catalog	4.1 centralized governance 4.5 row filters and column masks 4.6 ABAC 4.7 data classification.
Network security	3.2 IP access lists 3.3 PrivateLink 3.4 egress and exfiltration protection 3.9 context-based ingress.

Audit and monitoring	5.1 system tables 5.4 verbose audit logging 5.11 monitor requests for review.
Data protection	2.5 and 2.6 customer-managed keys 2.10 exfiltration-prevention settings.
AI commitments	AI 3.0 governed by Unity Catalog AI 4.0 access scoped to the user AI 1.0 stateless models AI 2.0 no training on your data.

Sensitive data disclosure via Model Providers

Concern	"what data goes to the model, where is it processed, and who can see retained data?"
Likelihood	Low
Impact	Moderate
Residual risk	Low
Bottom line	AI adds an inference hop, not a new class of accessible data.

Risk description

AI requests may include user inputs, metadata, sampled values, and prior results. Security teams worry about provider logs and cloud processing, and legal teams worry about data leaving the region. Only data the user is already authorized to see, plus their prompt, is sent. Row filters and column masks are applied first, so masked or filtered values never enter a prompt. Inference runs on zero-data-retention endpoints, so providers do not retain your data even for abuse monitoring, and neither we nor our partners train generative models on it. Processing stays within your residency boundary. Where cross-geo processing is enabled, the request goes through a Databricks-owned proxy and the destination region keeps no record. Retained operational data lives in the control plane, encrypted and eligible for customer-managed keys.

Control Category	Control
Data protection and encryption	2.5 CMK for managed services 2.6 CMK for storage 2.9 AES column encryption 2.10 exfiltration-prevention settings.
Unity Catalog	4.5 row filters and column masks, so masked values never enter a prompt 4.6 ABAC 4.1 centralized governance.

Network security	3.4 serverless Restricted egress 3.3 PrivateLink 3.6 serverless firewall.
Data residency	7.1 Databricks Geos, where AI features are Designated Services.
Audit and monitoring	5.1 system tables 5.4 verbose audit logging 5.11 monitor requests for review 5.2 personnel access controls.
AI commitments	AI 2.0 no training AI 1.0 stateless models AI 5.0 encrypted everywhere with CMK AI 7.0 data residency AI 6.0 SDLC and tenant isolation.

Prompt injection, jailbreaks, or malicious retrieved content

Concern	"can hostile content tell Genie to ignore instructions or leak data?"
Likelihood	Moderate
Impact	Low
Residual risk	Low
Bottom line	The decisive control is not the model's compliance but the permission boundary. A successful injection reaches exactly what the user could reach by asking directly.

Risk description

Instructions can be embedded in user input or in retrieved content, such as a table value, a notebook output, or a connector result, intended to manipulate the model, reveal context, or steer an action. Hostile content can influence what Genie says, but it cannot expand what Genie can access or do. On manipulation, model-provider content filtering plus Databricks hardening are validated by a continuous internal AI red team. On data, even a successful injection cannot return data beyond the user's entitlements, because Unity Catalog is enforced at the query engine under the caller's identity, not by the model's cooperation. On actions, Genie's actions are SQL under the user's grants, and tools and connectors are governed as Unity Catalog securables.

Control Category	Control
Unity Catalog	4.5 row filters and column masks, enforced regardless of what the model was told 4.6 ABAC 4.7 data classification.

Identity and access	1.7 least privilege caps the blast radius 1.11 user-isolation compute.
Audit and monitoring	5.11 monitor requests for review 5.9 trusted and reputable sources only 5.4 verbose audit logging.
Network and data protection	3.4 egress controls block steered exfiltration 2.10 exfiltration-prevention settings.
Connections and AI services	6.2 MCP tools as UC securables 6.3 AI tools as UC functions 6.4 external HTTP with OAuth.
AI commitments	AI 8.0 protection from harmful output AI 9.0 tested by our AI red team AI 3.0 governed by Unity Catalog AI 4.0 access scoped AI 1.0 stateless.

External connector and shadow-AI risk

Concern	"can users quietly wire enterprise sources into AI without the security team knowing?"
Likelihood	Low, and Moderate where connection governance is not applied
Impact	Moderate
Residual risk	Low
Bottom line	Two settings decide your residual risk. The first is who can create connections. The second is whether serverless egress is set to Restricted.

Risk description

Genie can connect to external systems, including data sources, MCP servers, Unity Catalog functions used as tools, and external HTTP. The shadow-AI concern is a sensitive source wired into AI without policy or visibility.

A connection is not a quiet side-channel. Every one is a Unity Catalog securable. Creating one requires explicit privilege, using one is governed by grants, tool selection limits what is exposed, service policies allow or deny individual calls, and both creation and every invocation are audited. Unity AI Gateway governs MCP servers centrally and provides managed OAuth for external MCP servers. With serverless Restricted egress, outbound reach is confined to destinations defined in Unity Catalog locations and connections.

Control Category	Control
Connections and AI services	6.1 third-party connections as UC securables 6.2 MCP services as UC securables 6.3 AI tools as UC functions 6.4 external HTTP with managed OAuth.
Identity and access	1.7 restrict who can create connections 1.4 and 1.6 limit admins 1.12 governed service credentials.
Network security	3.4 serverless Restricted egress 3.6 serverless firewall 3.3 PrivateLink.
Audit and monitoring	5.1 creation and invocation logged 5.11 Unity AI Gateway centralizes MCP monitoring 5.3 cloud logs.
Unity Catalog and data protection	4.1 central governance 4.3 tags 2.10 exfiltration prevention.
AI commitments	AI 3.0 governed by Unity Catalog AI 4.0 access scoped to the user AI 6.0 SDLC and tenant isolation.

Static sharing and downstream disclosure of results

Concern	"if a privileged user shares an answer, is the viewer's permission rechecked?"
Likelihood	Low to Moderate
Impact	Moderate
Residual risk	Low in-platform, and Low to Moderate for static exports
Bottom line	Keep the recheck in force. Share the Agent, dashboard, or table rather than an export. Limit static-artifact creation with exfiltration-prevention settings. Use governed sharing, Delta Sharing or Clean Rooms, when data must reach another party. Once content is a screenshot or an emailed file, your downstream controls own it. That is a shared-responsibility boundary, not a platform gap.

Risk description

An answer, dashboard, query result, or export can be shared outside the original access path, as live access or as a static artifact. A static artifact carries whatever the sharer could see when it was created. It depends on how it is shared, and we are direct about the boundary. Live, in-platform sharing is rechecked. Share a Genie Agent, a live dashboard, or table access, and the viewer's own Unity Catalog

permissions are enforced at view and query time. Row filters and column masks apply per viewer, and a less-privileged viewer does not inherit the sharer's access. Static artifacts are not rechecked, by nature. A screenshot, a CSV or PDF export, or copied text has left the governance boundary, and there is no longer a query to govern. This is not unique to AI, since any report behaves the same way, but AI makes shareable answers faster to produce.

Control Category	Control
Data protection and encryption	2.10 exfiltration-prevention to disable download, export, and clipboard 2.7 Delta Sharing for governed downstream sharing 2.8 recipient token lifetime 2.11 Clean Rooms to share insight rather than a copy.
Unity Catalog	4.5 row filters and masks, never materialized 4.6 ABAC 4.3 tags 4.4 lineage to track where data went.
Audit and monitoring	5.1 system tables, where exports and queries are logged 5.4 verbose audit logging.
Identity and network	1.7 least privilege for a smaller export footprint 3.4 egress controls to bound destinations.
Connections and Geos	6.1 onward sharing governed as a securable 7.1 residency, so shared data stays in region.
AI commitments	AI 4.0 access scoped, so live views are rechecked per viewer AI 3.0 governed by Unity Catalog AI 5.0 encrypted with CMK for retained artifacts.

AI supply chain, model routing, and content integrity

Concern	"can users choose arbitrary models, bypass approved routes, or introduce untrusted tools?"
Likelihood	Low, and Moderate where model routing and supply-chain controls are not applied
Impact	Moderate
Residual risk	Low
Bottom line	Every model, route, tool, and MCP server is a permissioned, audited object. Regardless of which is used, data access stays within the user's Unity Catalog grants.

Risk description

AI introduces a new supply chain, including models, provider routes, prompt templates, tools, MCP servers, and upstream data, each a potential integrity target.

Each part is a governed, permissioned, audited object rather than a free choice. The managed Genie experience runs on vetted providers, Azure OpenAI, Anthropic, or Databricks-hosted models, not models a user picks at will. Where the platform does expose model choice, in Model Serving and agents, Unity AI Gateway is the control point. Routing through Unity AI Gateway makes it the single governed path, with centralized access control, guardrails, payload logging, and audit. Tools enter only as Unity Catalog securables, with managed OAuth for external MCP servers. Upstream, trusted-source controls, model scanning, library allowlists, and Data Quality Monitoring guard models, libraries, and data.

Control Category	Control
Audit and monitoring	5.11 Unity AI Gateway for route governance and guardrails 5.9 trusted and reputable sources with scanning of untrusted models 5.8 library allowlists 5.10 Data Quality Monitoring for poisoning and drift.
Connections and AI services	6.2 MCP services as UC securables 6.3 AI tools as UC functions 6.1 third-party connections as securables 6.4 external HTTP with managed OAuth.
Identity and access	1.7 restrict who can register models and routes 1.4 and 1.6 limit admins 1.10 service principals for production config.
Unity Catalog and network	4.1 central governance 4.6 ABAC 3.4 egress controls 3.3 PrivateLink 2.10 exfiltration prevention.
Data residency	7.1 routing and processing stay in region.
AI commitments	AI 9.0 the red team feeds model onboarding AI 6.0 SDLC and tenant isolation AI 8.0 protection from harmful output AI 1.0 and AI 2.0 stateless and no training AI 3.0 and AI 4.0 UC-scoped.

Answer accuracy, hallucination, and overreliance

Concern	"can I trust what Genie tells me, or will it confidently give a wrong
----------------	---

	answer that someone acts on?"
Likelihood	Moderate
Impact	Variable, Low for exploratory use and High if an unverified answer drives a high-stakes decision.
Residual risk	Low to Moderate, with verification.
Bottom line	A wrong answer is a quality issue, not a disclosure. The data behind it is correctly scoped and the query is inspectable, so errors are catchable before they are trusted.

Risk description

This comes up as often as any security question, but it is a product-quality question rather than a confidentiality one, which is why it is assessed here.

Language models can produce plausible but incorrect output. A natural-language-to-SQL assistant may misinterpret a question, generate SQL that computes the wrong thing, or summarize results inaccurately, and a user may act on it without verifying.

Genie's answers are grounded, not generated. They are computed by SQL against your governed tables and executed under Unity Catalog, so they are real query output scoped to the user, not a narrative the model invented. Genie is also transparent by design. It shows the generated SQL for every answer, so you or a data steward can inspect exactly how it was computed, and flawed logic is visible and checkable before anyone relies on it. Answers are curated and measured. Agent authors supply metadata, instructions, example queries, and trusted assets, and accuracy can be benchmarked against known-good question-and-answer pairs and tracked over time. In-product feedback surfaces questions Genie answers poorly.

The residual risk is overreliance, acting on an unverified answer. That is a usage discipline as much as a control. Read the SQL, and verify before a high-consequence decision.

The failure modes bound as follows. Fabricated numbers are not the failure mode, because data answers are computed by SQL against your governed data, not recalled from the model, under AI 3.0 and AI 4.0. Wrong interpretation is catchable because Genie shows the SQL, control 8.9, and reduced by grounding through instructions, example and certified queries, and trusted assets, controls 8.5, 8.6, and 8.8. Bad underlying data is guarded by Data Quality Monitoring against drift, poisoning, and anomalies, control 5.10. Overreliance is the residual risk, mitigated by reading the SQL and verifying high-stakes results, and by benchmarking so accuracy is measured rather than assumed, control 8.10. Accountability is served because questions and answers are retained for review through request monitoring and audit logs, controls 5.11 and 5.4.

Control Category	Control
Genie accuracy and trust	8.9 inspect the generated SQL 8.5 example SQL queries 8.8 certified trusted assets 8.2 knowledge store

	8.1 annotated metadata 8.6 instructions 8.3 join relationships 8.4 SQL expressions for metrics 8.7 scope and pre-join with views 8.10 benchmark accuracy over time 8.11 monitor usage and act on feedback.
Unity Catalog	4.1 centralized governance 4.4 lineage to trace the data behind an answer 4.3 tags 4.7 data classification.
Audit and monitoring	5.10 Data Quality Monitoring, because an answer is only as good as its data 5.11 monitor requests for review 5.1 system tables 5.4 verbose audit logging.
AI commitments	AI 9.0 tested by our AI red team AI 8.0 protection from harmful output AI 3.0 and AI 4.0 answers computed only from real, authorized data.

Control catalog

Categories 1 through 7 are the standard Databricks platform controls. The full catalog — each control with its shared-responsibility split, the customer action, and how to validate it — is maintained in the [Controls for Genie matrix](#). They are summarized by category below; category 8, Genie accuracy and trust, is Genie-specific and detailed in full after it.

1. Identity and access (14): SSO/MFA, AIM/SCIM sync, least privilege, token management, and the Genie interface as a least-privilege control.
2. Data protection and encryption (11): bucket hygiene, customer-managed keys, Delta Sharing, AES column encryption, and exfiltration-prevention settings.
3. Network security (11): customer-managed VPC, IP access lists, PrivateLink, egress controls, and context-based ingress.
4. Unity Catalog (7): centralized governance, row filters and column masks, ABAC, and data classification.
5. Audit and monitoring (11): system tables, CloudTrail, verbose audit logging, Data Quality Monitoring, and request review.
6. Connections and AI services (4): third-party connections, MCP servers, and AI tools governed as Unity Catalog securables, plus external HTTP with OAuth.
7. Databricks Geos (1): data-residency boundaries.

Genie accuracy and trust

Curate and verify each Genie Agent so answers stay grounded in your governed data and are trustworthy. Controls 8.1 through 8.7 curate and ground the Agent. Controls 8.8 through 8.11 verify and improve it.

- **8.1 Provide well-annotated table and column metadata.** Genie generates SQL from your table and column names, comments, and descriptions, so clear, business-friendly metadata is the foundation of answer quality. Add synonyms so Genie recognizes the terms your users actually type.
- **8.2 Curate the Genie knowledge store.** The curated set of semantic definitions, including agent-level metadata, synonyms, instructions, and structured SQL, that steers how Genie interprets questions for your domain.
- **8.3 Define join relationships.** Specifies how tables combine so Genie generates correct joins rather than inferring them, improving accuracy on multi-table questions.
- **8.4 Define SQL expressions for business metrics.** Captures business logic as reusable measures, filters, and dimensions so metric definitions are computed consistently across answers.
- **8.5 Add example SQL queries.** Shows Genie the correct query pattern for common or ambiguous questions. Example queries are among the most effective ways to improve accuracy, and are preferred over plain-text instructions.
- **8.6 Add plain-text instructions.** Conveys company-specific jargon, business logic, and formatting preferences not captured in the schema. This complements, rather than replaces, example queries.
- **8.7 Scope the Agent and pre-join with views.** Keeps each Agent focused on only the tables needed, and pre-joins complex topics into views or metric views so Genie reasons over a simpler, curated surface.
- **8.8 Certify trusted assets and verified answers.** Marks parameterized example queries and Unity Catalog SQL functions as trusted, so Genie returns a certified verified answer for anticipated questions rather than generating new SQL.
- **8.9 Inspect the generated SQL.** Genie shows the exact SQL behind every answer, so users and stewards can verify how a result was computed before trusting it.
- **8.10 Benchmark accuracy over time.** Evaluation questions paired with known-good answers let you measure accuracy, catch regressions after configuration changes, and track quality over time.
- **8.11 Monitor usage and act on feedback.** The Monitor tab, weekly digest, and in-product user feedback surface questions Genie answers poorly, so the knowledge store can be improved continuously.

AI-specific commitments

These are guarantees we enforce on every Genie interaction, independent of your configuration.

- **AI 1.0 Models are stateless.** The model stores no data between requests. Even in long conversations, each request carries the conversation so far, and nothing persists in the model afterward. This is a property of the models themselves, not a Databricks-specific configuration.
- **AI 2.0 No training on your data.** Partner-powered features use zero-data-retention endpoints. Model partners do not retain data you submit, even for abuse monitoring, and we do not train generative foundation models on it.
- **AI 3.0 Governed by Unity Catalog.** One permission model, enforced end to end. Generated SQL runs against your data with each user's own Unity Catalog permissions applied to the results.
- **AI 4.0 Access scoped to the user.** Genie only ever uses data the user can already access. Information the current user cannot access is never sent to the models, and a question about data

they cannot access returns an empty response. Row filters and column masks are enforced per user.

- **AI 5.0 Encrypted everywhere with CMK available.** Encrypted at rest at AES-256 or greater and in transit at TLS 1.2 or greater. Long-term storage, including the Genie Agents database, dashboards, and history, is covered by customer-managed keys for managed services.
- **AI 6.0 SDLC and tenant isolation.** Built and operated under the same secure development and delivery lifecycle as the rest of the platform, including penetration testing and internal access controls.
- **AI 7.0 Data residency.** AI features are Designated Services and comply with data residency boundaries. Where cross-geo processing is enabled, the request goes through a Databricks-owned LLM proxy and the destination region keeps no logs, state, or record.
- **AI 8.0 Protection from harmful output.** Azure OpenAI content filtering and Anthropic safety mechanisms, plus Databricks hardening, validated across thousands of simulated interactions covering harmful content, jailbreaks, insecure code generation, and third-party copyright content.
- **AI 9.0 Tested by our own AI red team.** Our internal AI red team continuously tests Genie against real attack techniques, including prompt injection, jailbreaks, insecure code generation, and harmful content. Findings feed back into guardrails, output handling, and model onboarding. A written report is available under NDA.

More Information

For more information about Databricks security, please see our [Security and Trust Center](#). The [Security and Trust Overview White Paper](#) will provide you a quick glimpse of our security controls, and you can find more detail within our [product documentation](#) and in our Enterprise Security Guide, along with compliance reports like our SOC 2 Type II. Your Databricks representatives will be happy to assist you with any resources you need.

Revision history

Date	Author	Description of changes	Version #
2026 Jul 20	Arun Pamulapati	Initial version	1.0